02 Chisel 与工具链

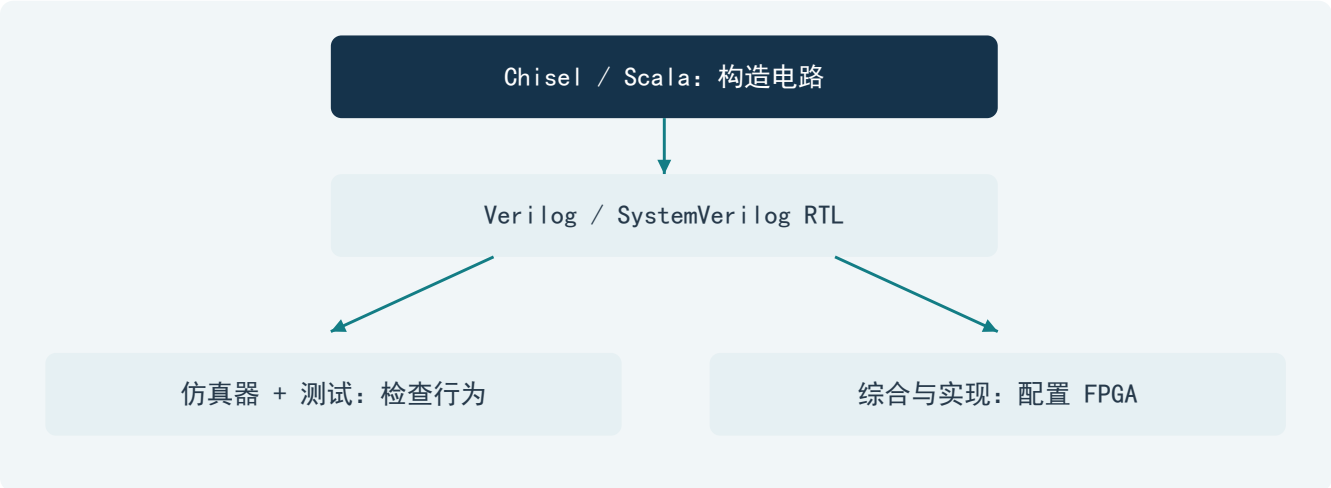
先把工具名字分清，再开始安装

会议前面写了 Trisim、Trisol，后面明确提到 Chisel 和 Chipyard。本资料按 Chisel 准备入门学习，这一名称推断仍待导师核实。

名称	是什么	你本周需要理解什么
RTL	一种硬件描述层次	状态如何在寄存器间流动
Verilog / SystemVerilog	硬件描述语言	描述接口、逻辑与寄存器
Chisel	基于 Scala 的硬件构造语言	用程序构造电路，生成 RTL
Scala	一种编程语言	读懂本册用到的少量语法
Icarus / Verilator	仿真相关工具	在计算机里检查 RTL 行为
Chipyard	集成多种生成器和工具的 SoC 研究框架	知道它在后续系统实验中的位置
FPGA	可配置逻辑硬件	后期把设计实现到真实器件上

SoC 是 System on Chip，片上系统，可包含处理器、内存接口、总线和外设。FPGA 可以实现这样的数字设计，但生成 RTL 之后仍需要综合、布局布线与板级验证。

最小工具链



Chisel 不是把任意 Scala 算法自动变成硬件。你要使用 Chisel 提供的构件，明确描述寄存器、连接与计算关系。[C1][C5]

2.1 只学够用的 Scala

六种写法就能读入门模块

写法	通俗解释	小例子
val	给一个值起名，名字不能重新指向别的值	val width = 4
class	定义一种类型；本册用来定义模块	class Gate extends Module
extends	基于已有类型扩展	extends Module
new	创建一个对象	new Bundle
{ }	把一组内容放在一起	模块、接口或条件块
import	使用库中的名字	import chisel3._

容易困惑的两种赋值

```scala
val limit = 4
val stopped = RegInit(false.B)
stopped := true.B
```

第一行创建一个 Scala 名字 limit，值为 4。第二行在构造电路时创建一个寄存器，并用 stopped 指向它。第三行使用 Chisel 的连接运算符 :=，描述这个硬件寄存器的更新输入。

所以 val stopped 并不表示“寄存器以后永远不能改变”。不可重新绑定的是 Scala 名字，硬件信号的值仍然可以随电路运行而变化。

Scala 数值与硬件数值

4 是 Scala 里的整数，可用于决定电路宽度。4.U 表示无符号硬件常量。4.W 表示位宽为 4。false.B 表示硬件布尔常量假。

```scala
val count = RegInit(0.U(4.W))
```

从里向外读：创建值为 0、宽度为 4 的无符号硬件常量，再创建以它为复位值的寄存器。

练习：val width = 8 与 val x = Wire(UInt(8.W)) 分别决定“构造电路的参数”和“电路中的信号”中的哪一个？

2.2 第一个 Chisel 组合模块

还是同一个二选一选择器

```scala
import chisel3._

class Mux2 extends Module {
  val io = IO(new Bundle {
    val a = Input(Bool())
    val b = Input(Bool())
    val sel = Input(Bool())
    val y = Output(Bool())
  })
  io.y := Mux(io.sel, io.b, io.a)
}
```

Module 是模块的基础类型。IO 定义模块边界上的端口，Bundle 把多个端口组织成一组，Bool() 表示 1 位布尔类型，Input / Output 说明方向。

io.y 表示访问 io 里的 y。Mux(条件, 真时输入, 假时输入) 对应 Verilog 的三目运算符。因此最后一行是 sel=1 时输出 b，否则输出 a。

把两种写法对起来

Chisel	Verilog 中相近的描述
Input(Bool())	input wire a
Output(Bool())	output wire y
io.y := Mux(io.sel, io.b, io.a)	assign y = sel ? b : a
UInt(4.W)	4 位无符号向量

这里只比较行为与接口的对应关系，生成的 RTL 名字和排版未必与手写代码一致。优化工具也可能化简逻辑。

“写了什么”与“生成了什么”

写 Chisel 时，你在执行一个构造电路的 Scala 程序；生成结果则是硬件描述。运行构造程序时执行的 println，只是在电脑上打印文字。电路上的运行时打印需要专门的仿真支持，不能混为一谈。

练习：假设 a=1、b=0、sel=1，y 是多少？这里需要时钟吗？

2.3 寄存器与条件更新

将 RTL 计数器翻译成 Chisel

```scala
import chisel3._

class Counter4 extends Module {
  val io = IO(new Bundle {
    val enable = Input(Bool())
    val count = Output(UInt(4.W))
  })
  val count = RegInit(0.U(4.W))
  when(io.enable) {
    count := count + 1.U
  }
  io.count := count
}
```

RegInit 表示带复位初值的寄存器。标准 Module 提供时钟与复位接口。when 用来描述硬件条件；没有命中更新条件时，这个寄存器保持原值。[C4]

本例将加法结果接回 4 位寄存器，超出位宽的高位无法保留，所以会出现与 01 中相同的模 16 计数行为。若需要显式保留进位，必须设计更宽的输出或寄存器。

Scala if 与 Chisel when 的区别

Scala 的 if 通常在构造电路时决定生成哪种结构，例如是否生成一个调试模块；Chisel 的 when 由电路运行时的硬件信号决定某条连接如何选择。

```scala
val width = 4
val count = RegInit(0.U(width.W))
when(io.enable) { count := count + 1.U }
```

width 在生成时决定寄存器有几位；enable 在电路运行时决定是否更新。这两个“时间”应当分开理解。

同步复位的理解

教学对应图按普通同步复位理解，具体生成的复位类型以模块及生成结果为准。Chisel 还支持异步与抽象复位类型；本周只需知道复位语义应当明确，不能只看变量名猜测。[C3]

练习：这个模块生成完成后，修改 Scala 里的 width 会立刻改变已经配置好的 FPGA 吗？需要经过哪些步骤？

2.4 实验准备：固定一组教学依赖

在已有环境里做最小实验

配套 chisel_project.scala 固定使用 Scala 2.13.16 与 Chisel 7.2.0，目的是让示例可复现，不表示这些是最新版本。编译器插件版本与 Chisel 版本配套，Scala 完整版本也要匹配。

```scala
//> using scala "2.13.16"
//> using dep "org.chipsalliance::chisel:7.2.0"
//> using plugin "org.chipsalliance:::chisel-plugin:7.2.0"
//> using options "-language:reflectiveCalls"
```

这些以 //> 开头的行是 Scala CLI 指令，告诉工具下载什么依赖。普通 // 则是代码注释。第一次运行需要网络、下载依赖及 Java 环境，后续通常可利用缓存。

安装入口

Scala CLI 官方安装页提供 Windows 安装器、包管理器和 Linux 入口。建议使用已经熟悉的平台，完整入口见 [C6]。Windows 若有 WinGet，可在 PowerShell 使用官方列出的命令：

```powershell
winget install virtuslab.scalacli
```

安装后重新打开终端，再检查：

```text
scala-cli version
java -version
```

Scala CLI 可以管理所需 JDK。若版本或下载报错，先参考官方安装页，不必同时安装 SBT、Mill 和完整 Chipyard。

先做官方烟雾测试，可选

官方 Chisel 安装页提供 chisel-example.scala 单文件示例。下载后执行 scala-cli chisel-example.scala，可以先验证网络、依赖与基础工具链。[C2]

> Chisel 在 Windows 上的后端工具支持可能比纯 Scala 程序复杂；若生成后端报平台问题，可以转到已有的 WSL 环境。资料中的 Windows Python 练习不依赖这一步。

2.5 生成自己的安全门 RTL

完整模块已经放在配套文件里

SafetyGate.scala 中最核心的部分如下；完整文件还包含 import、输入输出定义和生成入口。

```scala
val stopped = RegInit(false.B)
when(io.killReq) {
  stopped := true.B
}
io.stopped := stopped
io.allow := io.request && io.authorized && !stopped
```

这段代码与 01 的停止门有相同的主要状态行为：复位清除停止位，采样到停止请求后保持停止，allow 由请求、授权和停止位共同决定。

在配套目录运行：

```bash
scala-cli run chisel_project.scala SafetyGate.scala --main-class EmitSafetyGate
```

入口中的 ChiselStage.emitSystemVerilog 把电路生成成 SystemVerilog 字符串，再打印到终端。示例关闭了后端随机初始化附加代码，便于用 Icarus 检查；寄存器仍应按规范复位。结果应包含 clock、reset、请求与授权端口、stopped 寄存器及 allow 逻辑。

不要只找“一模一样的一行代码”

后端可能重命名内部信号或把条件更新化简。比较时检查：输入输出方向、寄存器初值、何时更新、未更新时是否保持、组合输出规则。这些比格式更重要。

必做推演

先复位，再输入请求与授权，再采样 killReq=1，最后撤回 killReq。把每一步 stopped 和 allow 的值写下来，并与 01 的逐拍表对照。

生成与仿真是两个步骤

成功打印 RTL 后，还需测试平台检查行为。配套提供 SafetyGate.generated.sv，是从本例实际生成结果中提取的主模块；适配文件把端口接到 01 的同一套测试上。示例说明给出运行命令与验证范围。

2.6 常见报错与小练习

按报错发生的位置排查

现象	常见原因	先做什么
找不到 scala-cli	未安装或 PATH 未更新	重新开终端，执行 version
dependency 下载失败	网络、仓库访问或版本错误	核对固定依赖，保留完整报错
提示 compiler plugin	插件缺失或版本不配套	同时使用 chisel_project.scala
把 Bool 用到 Scala if 出错	混淆构造时参数与硬件条件	对硬件条件使用 when
输出未连接	没有给所有输出路径定义值	从模块接口逐个检查
firtool / 平台错误	RTL 生成后端不可用或不匹配	查官方安装与平台支持

练习 A：授权门

给一个已有模块的输出接上规则：“有请求、授权有效、且没有暂停”。只写最后一行 Chisel 连接，端口名为 request、authorized、paused、allow。

练习 B：故意找区别

比较下面两段：

```scala
val x = Wire(Bool())
x := io.killReq
```

```scala
val x = RegInit(false.B)
when(io.killReq) { x := true.B }
```

第一段与第二段是否都有记忆？killReq 撤回后分别会怎样？

练习 C：停止优先级

已有同步复位模块默认复位会清除停止位。如果研究要求“停止优先于恢复”，你应先修改什么规范、再检查哪些输入组合？允许写自然语言，不要求本周实现复杂复位结构。

2.7 参考答案与工具链复习

本册练习答案

width=8 是构造电路的参数；Wire(UInt(8.W)) 创建 8 位硬件信号。二选一选择器在 a=1、b=0、sel=1 时输出 0，不需要时钟。

修改生成器参数后，要重新运行生成、综合与实现，并重新配置 FPGA，才能改变对应硬件结构。具体还需看工程构建流程。

练习 A：

```scala
io.allow := io.request && io.authorized && !io.paused
```

练习 B：Wire 直接跟随 killReq，不保存历史；寄存器版本在采样到 1 后记住它，命令撤回也不会自动解除。

练习 C：先定义停止与恢复的优先级、谁有权恢复、复位阶段是否允许输出；再检查同时到来的停止和恢复、复位期间请求、停止命令撤回、连续重复停止等情况。不要仅靠名字叫 reset 就默认它可信。

本周对 Chipyard 的学习深度

知道 Chipyard 将处理器生成器、外围模块与验证工具组织成系统平台即可。先能独立读小模块、生成 RTL、解释仿真，再研究将安全模块接入 SoC 的哪条路径。完整搭建 Chipyard 可留待平台与架构方向明确之后。[C5]

阅读来源

[C1] Chisel 官方介绍：https://www.chisel-lang.org/docs

[C2] Chisel 安装与后端说明：https://www.chisel-lang.org/docs/installation

[C3] 复位类型：https://www.chisel-lang.org/docs/explanations/reset

[C4] 时序电路：https://www.chisel-lang.org/docs/explanations/sequential-circuits

[C5] Chipyard 中的 Chisel：https://chipyard.readthedocs.io/en/stable/Tools/Chisel.html

[C6] Scala CLI 安装：https://scala-cli.virtuslab.org/install/

官方文档会更新。这里固定的教学版本用于示例复现；后续工程选型应核对导师平台与依赖兼容性。

制作验证：本例已在 Ubuntu / WSL 实际生成 RTL，并用 Icarus 通过与手写停止门相同的定向测试；未进行 FPGA 验证。详细环境见配套验证记录。